ESCALABILIDAD, DESACOPLAMIENTO Y EVOLUCIÓN DE SISTEMAS

Programación Web

Ing. Manuel Rojas

AGENDA



- Escalabilidad
- Escalamiento vertical
- Escalamiento horizontal
- Cuello de botella
- Acoplamiento
- Desacoplamiento
- Cambio localizado
- Evolución
- Compatibilidad
- Deuda técnica
- Sobrearquitectura

¿QUÉ ES ESCALABILIDAD?

Capacidad de un sistema para manejar un aumento de carga manteniendo un nivel aceptable de funcionamiento.

La carga puede aumentar por:

- Más usuarios
- Más solicitudes
- Más datos
- Más operaciones simultáneas
- Procesos más pesados.



ESCALABILIDAD ≠ RENDIMIENTO

Son conceptos relacionados, pero diferentes.

Rendimiento: qué tan rápido o eficientemente funciona el sistema actualmente.

Escalabilidad: cómo responde cuando aumenta la carga.

Un sistema puede responder en:
100 ms con 10 usuarios.

Pero con 5,000 usuarios:
8 segundos. Tenía buen rendimiento inicial, pero puede tener problemas de escalabilidad.



ESCALAMIENTO VERTICAL

Aumentar los recursos disponibles para una instancia.

Antes:
Servidor
4 CPU
8 GB RAM

↓

Después :
Servidor
16 CPU
32 GB RAM



También se conoce como: Scale Up

Ventajas

- Sencillo
- Pocos cambios arquitectónicos
- Fácil de administrar inicialmente.

Limitaciones

- Existe un límite físico
- Puede resultar costoso
- Seguimos dependiendo de una instancia.



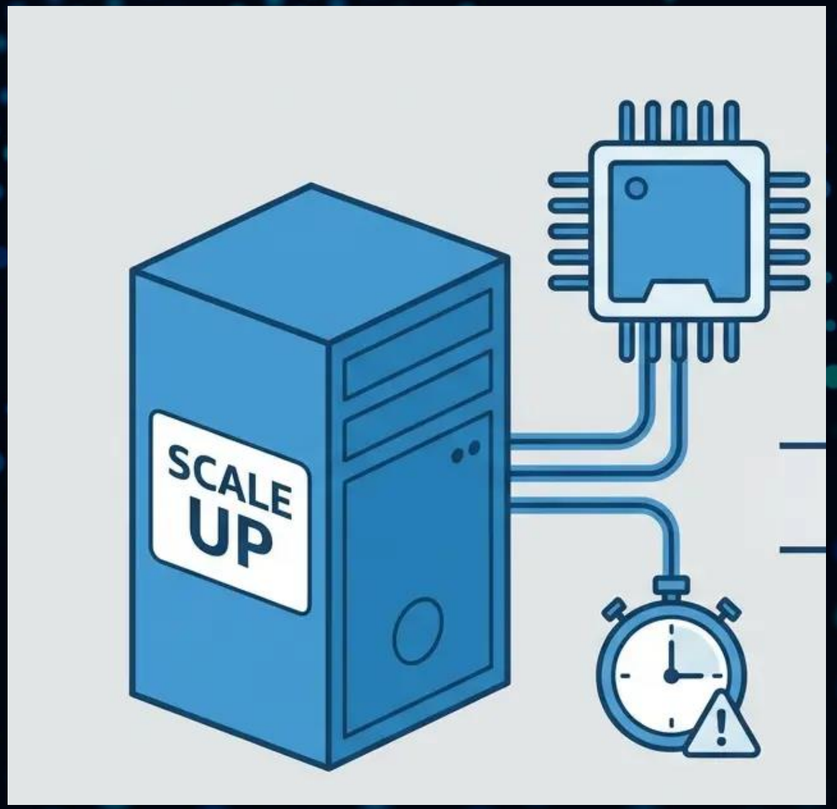
Ejemplo:

La base de datos comienza a quedarse sin memoria.

Podemos aumentar:

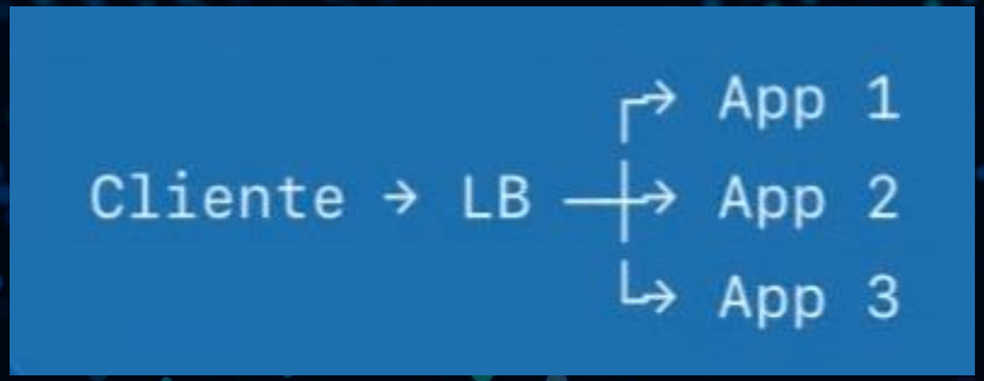
- RAM
- CPU
- almacenamiento

sin cambiar inmediatamente toda la arquitectura.



ESCALAMIENTO HORIZONTAL

Agregar más instancias capaces de atender la carga.
También: Scale Out



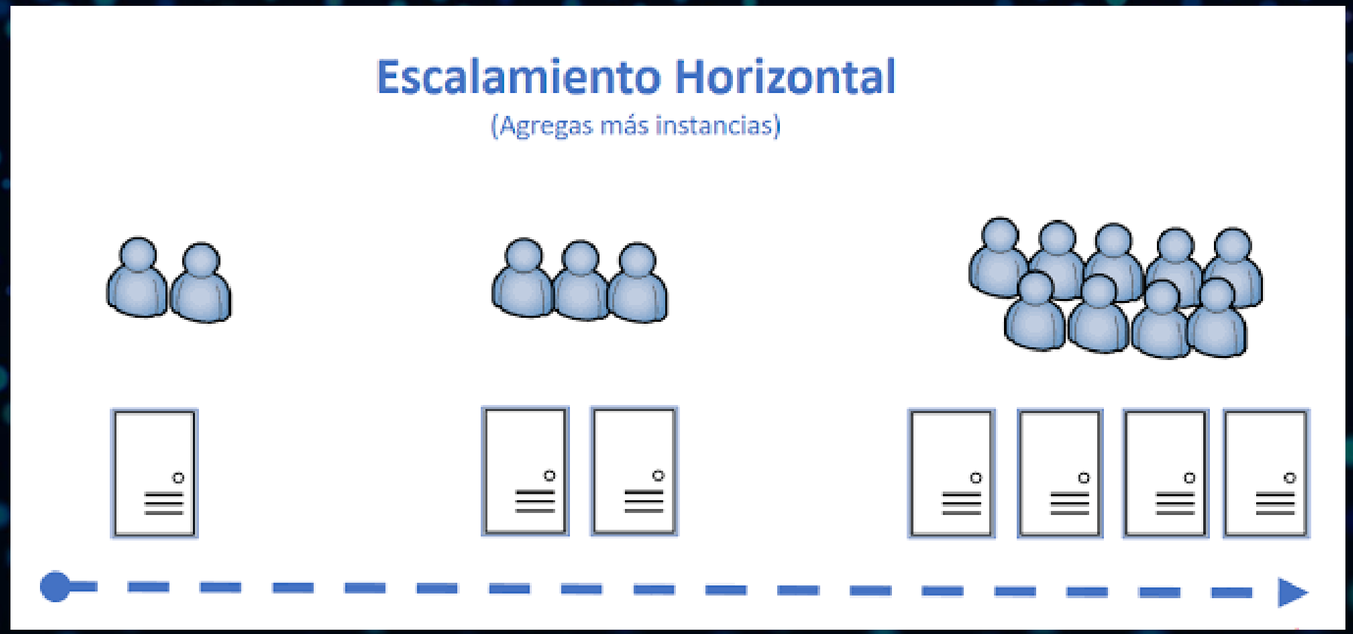
El Load Balancer distribuye las solicitudes.
En lugar de hacer una máquina cada vez más grande, agregamos más instancias.

Ventajas

- Permite distribuir carga
- Mayor capacidad de crecimiento
- Puede mejorar disponibilidad.

Desafíos

- Coordinación
- Estado compartido
- Sesiones
- Datos
- Infraestructura más compleja.



¿QUÉ DEBEMOS ESCALAR?

No necesariamente todo el sistema tiene el mismo problema.



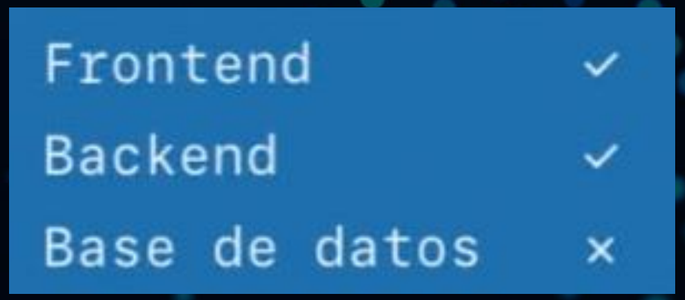
Si el problema está en Pedidos, aumentar recursos de todo el sistema puede ser innecesario.

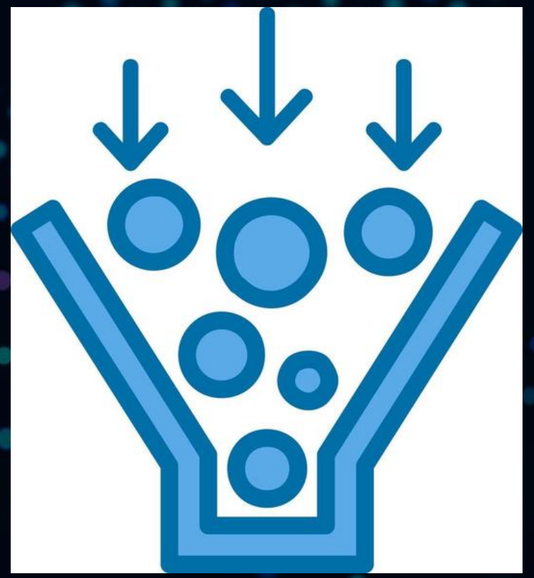
Una ventaja de ciertos diseños distribuidos es poder escalar componentes independientemente.Pero eso no significa que debamos crear microservicios únicamente porque algún día podríamos necesitar escalar.

CUELLO DE BOTELLA

Componente o recurso que limita el rendimiento o capacidad del sistema completo.
Puede estar en:

- Aplicación
- Base de datos
- Consultas
- Red
- Almacenamiento
- API externa;
- Procesamiento.





Agregar diez servidores backend no necesariamente resolverá una base de datos saturada.

DESACOPLAMIENTO: CRECER SIN CREAR UNA CADENA DE DEPENDENCIAS

Acoplamiento: Grado de dependencia existente entre componentes.

Ejemplo: A → B → C → D

Si cambiar D obliga a modificar C, luego B y finalmente A, tenemos un problema.

- Dependencia no significa automáticamente mal diseño.
- Los componentes de un sistema necesitan colaborar.
- El problema aparece cuando existe dependencia innecesaria o excesiva.



ALTO ACOPLAMIENTO

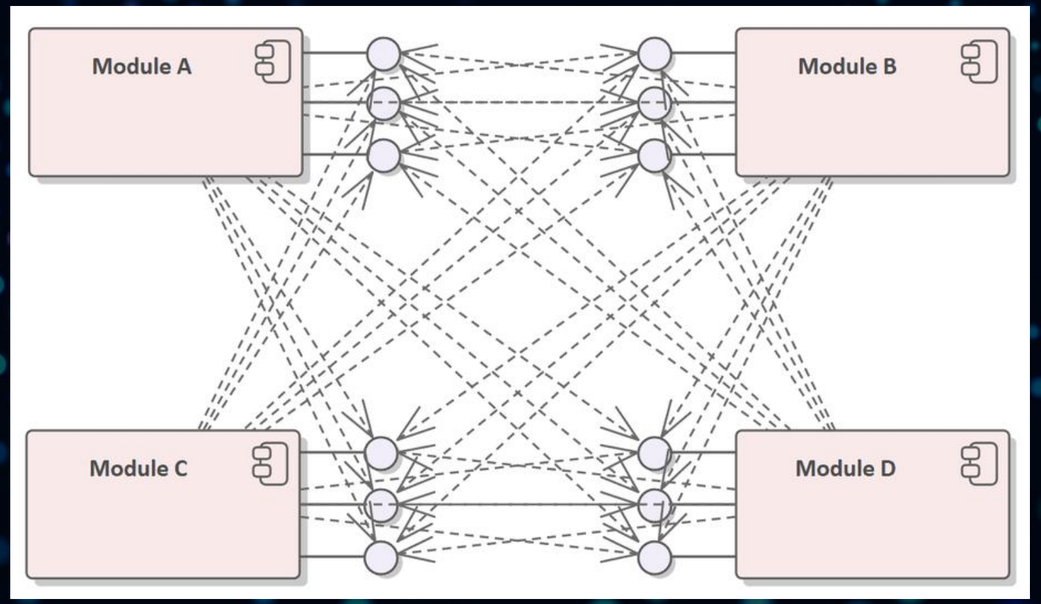
Tenemos alto acoplamiento cuando un componente conoce demasiado sobre otro.

Por ejemplo, Pedidos:

- conoce tablas internas de Inventario
- modifica directamente sus datos
- depende de detalles de implementación
- necesita conocer cada cambio interno.

PEDIDOS

- conoce tablas de Inventario
- conoce reglas de Inventario
- modifica Inventario directamente

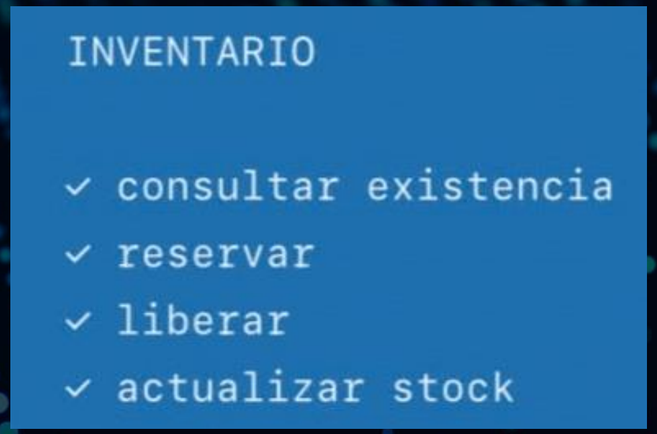


Un cambio pequeño puede propagarse por todo el sistema.

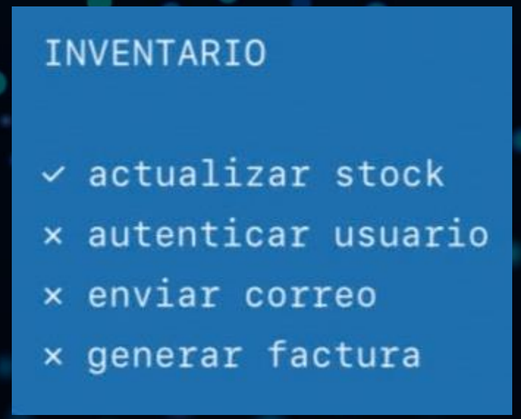
COHESIÓN Y ACOPLAMIENTO

Alta cohesión: Eesponsabilidades relacionadas permanecen juntas.
Bajo acoplamiento: Reducimos dependencias innecesarias entre componentes.

Ejemplo:



No:



DESACOPLAMIENTO MEDIANTE CONTRATOS

Una API puede funcionar como frontera.



Inventario puede cambiar internamente: Versión A, PostgreSQL
por:
Versión B
otra estructura interna

Si conserva el contrato esperado, Pedidos puede continuar funcionando.
Importante: Encapsular el cambio reduce su impacto.

DESACOPLAMIENTO TEMPORAL

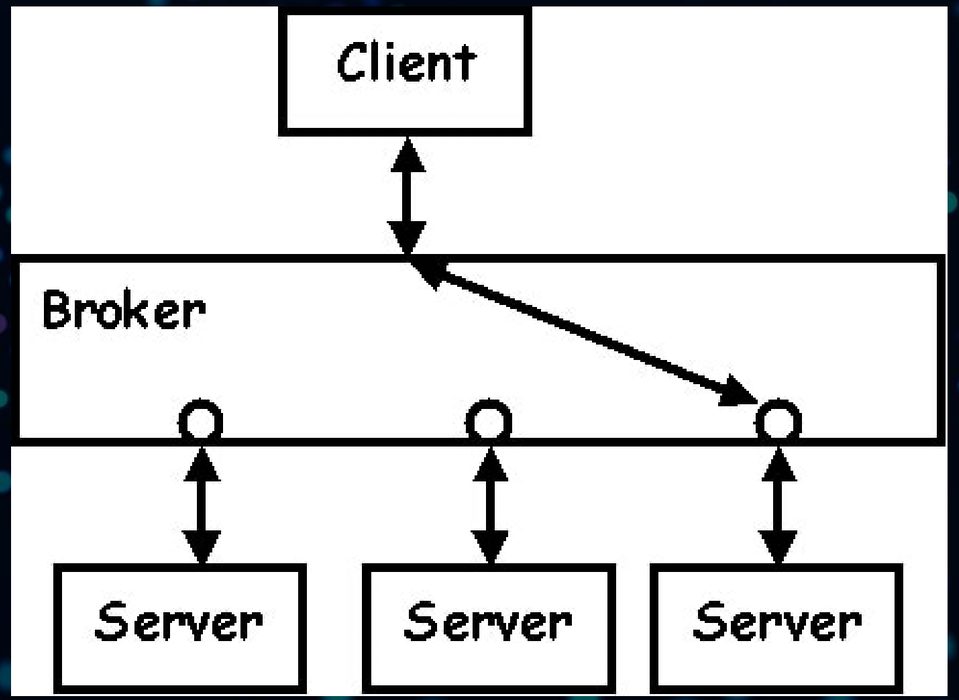
En comunicación síncrona:
A → espera → B

A depende temporalmente de que B responda.

Con eventos:
A → Evento → Broker → B

A puede continuar sin esperar necesariamente a B.
Esto puede reducir el acoplamiento temporal. Broker funciona como intermediario para facilitar la comunicación

Pero:
Asíncrono ≠ completamente desacoplado.
Aún existe un contrato del evento.



DISEÑAR PARA EL CAMBIO

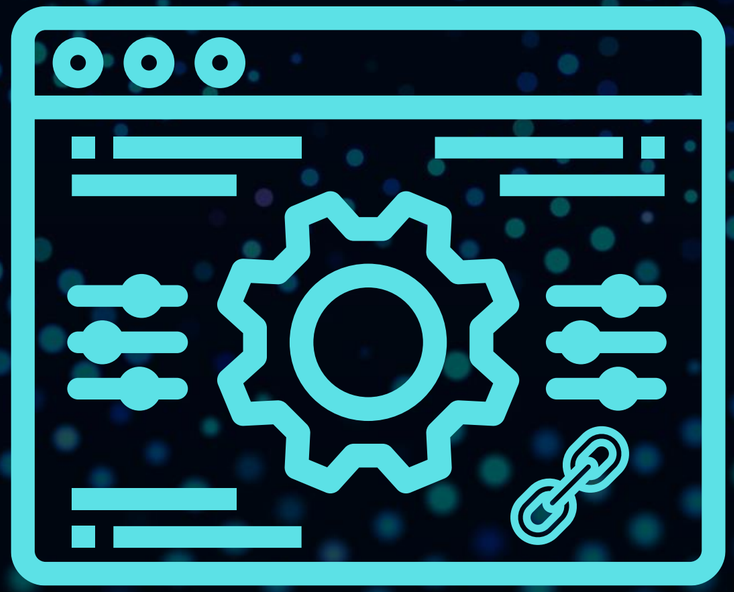
Esto no significa construir hoy todo lo que posiblemente necesitemos dentro de cinco años.

Eso podría llevarnos a:

- sobrearquitectura
- complejidad
- código innecesario
- mayor costo
- YAGNI.

Diseñar para el cambio significa:

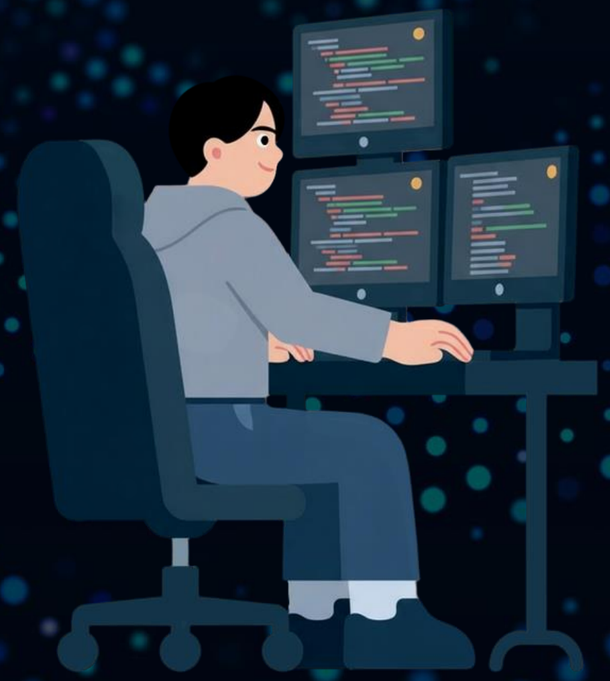
- Responsabilidades claras
- Contratos claros
- Módulos identificables
- Dependencias controladas
- Pruebas
- Documentación
- Capacidad de reemplazar componentes.



CAMBIO LOCALIZADO

Idealmente:
Cambio en Inventario
↓
Inventario

y no:
Cambio en Inventario
↓
Pedidos
↓
Pagos
↓
Reportes
↓
Frontend



Mientras más localizado pueda permanecer un cambio, más sencillo será evolucionar el sistema.

EVOLUCIÓN GRADUAL

No siempre debemos reemplazar todo el sistema.
Podemos evolucionar por etapas.

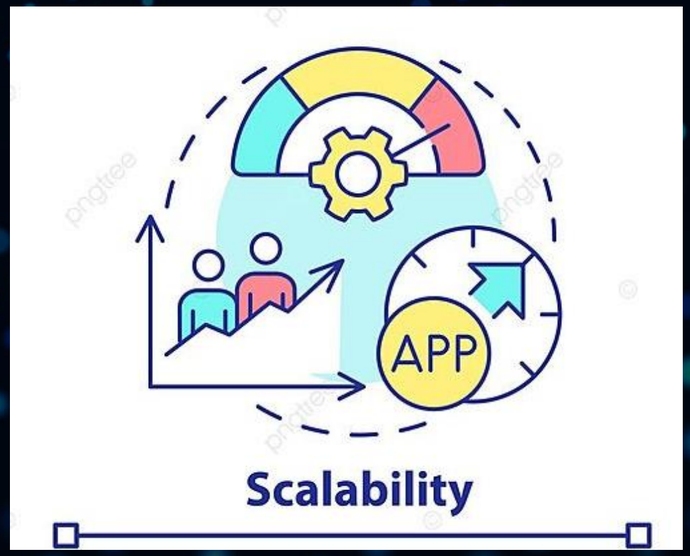
```
MONOLITO MODULAR
        ↓
identificamos un módulo con necesidad real
        ↓
separamos ese componente
        ↓
evaluamos
        ↓
continuamos solo si aporta valor
```

No necesitamos hacer: Monolito → 15 microservicios

ESCALABILIDAD TÉCNICA VS ORGANIZACIONAL

Escalabilidad técnica: Capacidad del sistema para soportar más carga.
Escalabilidad organizacional: Capacidad de varios equipos para trabajar sin bloquearse constantemente.

Ejemplo:
Si 30 desarrolladores necesitan modificar siempre: app.service.ts
tenemos un problema aunque el servidor tenga recursos suficientes.



SEÑALES PARA RECONSIDERAR LA ARQUITECTURA

Podemos empezar a revisar decisiones cuando aparecen situaciones como:

- Un componente necesita escalar mucho más que otros
- Equipos se bloquean constantemente
- Despliegues completos son demasiado riesgosos
- Determinados fallos afectan innecesariamente todo
- Un módulo cambia con mucha mayor frecuencia
- Límites del dominio están claramente definidos
- Una dependencia externa requiere aislamiento.

No significa: Apareció una señal → crear microservicio.
Significa: Apareció una razón para analizar nuestra decisión.



CUANDO UN SISTEMA COMIENZA A CRECER, PREGUNTARÍA:

1. ¿Qué está creciendo?
Usuarios, tráfico, datos, funcionalidades o equipo.

2. ¿Dónde está el problema?
Identificar el cuello de botella real.

3. ¿Qué componente necesita cambiar?
No asumir que debemos modificar todo.

4. ¿Qué dependencias tiene?
Analizar el impacto.

5. ¿Podemos resolverlo dentro de la arquitectura actual?
Primero considerar la opción más sencilla.

6. ¿Qué complejidad agregará nuestra solución?
Toda decisión tiene costo.

7. ¿Tenemos evidencia para justificarla?
No diseñar únicamente para escenarios imaginarios.



RETROALIMENTACIÓN

- Escalabilidad: Capacidad para soportar crecimiento manteniendo un funcionamiento aceptable.
- Escalamiento vertical: Aumentar recursos de una instancia.
- Escalamiento horizontal: Agregar más instancias.
- Cuello de botella: Elemento que limita la capacidad del sistema.
- Acoplamiento: Nivel de dependencia entre componentes.
- Desacoplamiento: Reducción de dependencias innecesarias.
- Cambio localizado: Modificación cuyo impacto permanece contenido.
- Evolución: Adaptación del sistema ante nuevas necesidades.
- Compatibilidad: Capacidad de evolucionar sin romper consumidores existentes.
- Deuda técnica: Decisión actual que puede aumentar el costo de cambios futuros.
- Sobrearquitectura: Introducir complejidad superior a la que el problema necesita.



GRACIAS POR SU ATENCIÓN